Optimización de PostgreSQL

LegalDataPlatform

Humberto Henriquez — Senior Data Engineer / Solutions Architect

Generado 2026-04-24

Optimización de PostgreSQL

Principios

1. **Medir antes de optimizar.** `pg_stat_statements` + `EXPLAIN (ANALYZE, BUFFERS)` guían toda decisión.
2. **El mejor índice es el que no existe.** Cada índice cuesta write-amplification y espacio. Se agregan sólo si hay un query real que los justifica.
3. **El planner es bueno, pero no mágico.** `ANALYZE` frecuente, statistics de muestras adecuadas y constraints honestas lo ayudan.
4. **Particionar antes de que duela.** Una tabla de 100M rows sin particionar es más dolorosa de particionar que una de 10M.

Configuración del servidor

Aplicada en `docker-compose.yml` para dev; para prod ir vía `postgresql.conf` o parameter groups:

Parámetro	Valor dev	Razón
`shared_buffers`	256 MB	~25% de RAM disponible
`effective_cache_size`	1 GB	estimación del OS cache disponible
`work_mem`	16 MB	memoria por operación sort/hash
`maintenance_work_mem`	128 MB	acelera VACUUM, CREATE INDEX
`max_connections`	200	+ PgBouncer para ratio
`shared_preload_libraries`	`pg_stat_statements`	tracking de queries
`log_min_duration_statement`	500 ms	log de slow queries

En producción, adicionalmente:
- `wal_compression = on` (ahorra WAL en cargas grandes).
- `checkpoint_timeout = 15min`, `max_wal_size = 8GB`.
- `random_page_cost = 1.1` en SSD.
- `autovacuum_vacuum_cost_limit = 2000` para no saturar.

Extensiones habilitadas

```
CREATE EXTENSION uuid-ossp;
CREATE EXTENSION pgcrypto;          -- gen_random_uuid()
CREATE EXTENSION pg_trgm;           -- trigram indexes
CREATE EXTENSION btree_gin;         -- composite GIN
CREATE EXTENSION pg_stat_statements;
CREATE EXTENSION unaccent;
```

Particionado

Tablas particionadas

- `legal_documents` — RANGE por `document_date`, mensual.
- `transactions` — RANGE por `transaction_date`, mensual.

Por qué particionar

Operación	Sin particionar	Particionado
Query "último mes" sobre 100M rows	full index scan, ~800ms	scan de una partición, ~40ms
`VACUUM`	toda la tabla, horas	por partición, minutos
`DELETE WHERE date < '...'`	horas, gran WAL	`DROP PARTITION`, O(1), sin WAL
Creación de índice	toda la tabla	por partición, paralelo

Mantenimiento

La función `create_monthly_partition(parent, date)` crea la partición del mes de `date`. Se debe correr mensualmente:

```
SELECT create_monthly_partition('transactions', CURRENT_DATE + INTERVAL '1 month');
SELECT create_monthly_partition('legal_documents', CURRENT_DATE + INTERVAL '1 month');
```

Programable vía:
- Cron dentro de Postgres con `pg_cron`.
- Prefect flow mensual.
- `pg_partman` (recomendado para producción).

Estrategia de índices

BTree compuesto — access path principal

```
CREATE INDEX ix_txn_counterparty_date
    ON transactions (counterparty_id, transaction_date);
```

Usado para "transacciones de una contraparte en rango de fechas". El orden importa: columna más selectiva primero.

BRIN — time-series columns

```
CREATE INDEX ix_txn_date_brin
    ON transactions USING BRIN (transaction_date)
    WITH (pages_per_range = 32);
```

BRIN usa **~10 KB por millón de rows** vs ~30 MB para BTree. Funciona porque las transacciones se insertan en orden temporal → las páginas están clusterizadas por fecha.

Partial indexes — filtros comunes

```
CREATE INDEX ix_contract_active
    ON contracts (counterparty_id)
    WHERE status = 'ACTIVE';
```

Reduce tamaño del índice al fragmento que realmente se consulta (~30% de las filas).

Covering indexes — evitar heap lookups

```
CREATE INDEX ix_txn_counterparty_recent
    ON transactions (counterparty_id, transaction_date DESC)
    INCLUDE (amount, currency)
    WHERE transaction_date ≥ CURRENT_DATE - INTERVAL '90 days';
```

El INCLUDE permite queries index-only scan → no hay que visitar el heap.

GIN con pg_trgm — búsqueda fuzzy

```
CREATE INDEX ix_legal_doc_title_gin
    ON legal_documents USING GIN (title gin_trgm_ops);
```

Soporta `ILIKE '%foo%'` y `similarity()` eficientemente. Sin este índice, esa query hace sequential scan.

GIN JSONB — queries de contención

```
CREATE INDEX ix_legal_doc_metadata_gin
    ON legal_documents USING GIN (metadata jsonb_path_ops);
```

`jsonb_path_ops` es 3× más rápido que el default para queries tipo `WHERE metadata @> '{"key": "value"}'`.

Materialized Views

`mv_monthly_revenue_per_counterparty` pre-computa agregados que son caros de calcular ad-hoc:

```
CREATE MATERIALIZED VIEW mv_monthly_revenue_per_counterparty AS ...
CREATE UNIQUE INDEX ux_mv_revenue_key
    ON mv_monthly_revenue_per_counterparty
    (counterparty_external_id, month, currency);
```

El índice UNIQUE es **requerido** para `REFRESH MATERIALIZED VIEW CONCURRENTLY`, que permite mantener lecturas durante el refresh.

Refresh strategies

Estrategia	Cuándo
Refresh completo CONCURRENTLY	Agregaciones estables (monthly revenue)
Refresh incremental (delta tables)	Datos near-real-time
Cron + Prefect	Cargas batch predecibles
Trigger-based	Datasets pequeños con cambios frecuentes

Query tuning workflow

1. Identificar el query sospechoso:

```
SELECT queryid, mean_exec_time, calls, query
FROM pg_stat_statements
ORDER BY mean_exec_time DESC LIMIT 20;
```

2. `EXPLAIN (ANALYZE, BUFFERS) <query>`. Buscar:
   - `Seq Scan` sobre tablas grandes → necesita índice.
   - `Nested Loop` con alto `actual rows` → considerar `Hash Join` (aumentar `work_mem`).
   - `Rows Removed by Filter` alto → índice parcial candidato.
   - `Heap Fetches` alto con Index Only Scan → VACUUM / visibility map.
3. Aplicar cambio, medir con query_benchmark.py.

Connection pooling

Topología

```
App (SQLAlchemy async) → PgBouncer (transaction mode) → Postgres
```

- App tiene pool de 10-20 conexiones a PgBouncer.
- PgBouncer mantiene pool de ~25 conexiones al backend.
- Ratio: 500 conexiones cliente : 25 backend = 20:1.

Reglas con PgBouncer transaction mode

- **No** usar `SET ... LOCAL` para sesiones largas.
- **No** usar prepared statements cacheados (deshabilitar `statement_cache_size=0` en asyncpg).
- **No** usar `LISTEN/NOTIFY` (requiere session mode).
- ✓ Transacciones cortas, sí.
- ✓ Read queries con pool, sí.

Usamos `direct_engine` (bypass PgBouncer) para:
- Migraciones Alembic.
- `COPY` bulk loads.
- Transacciones con locks (`SELECT ... FOR UPDATE`).

Mantenimiento programado

Cronjob diario (4 AM UTC):

```sql
-- 1. VACUUM + ANALYZE de tablas grandes
VACUUM ANALYZE transactions_$(date +%Y_%m);
VACUUM ANALYZE legal_documents_$(date +%Y_%m);

-- 2. Refresh MViews
CALL refresh_analytical_views();

-- 3. Asegurar particiones del mes siguiente
SELECT create_monthly_partition('transactions', CURRENT_DATE + INTERVAL '1 month');
SELECT create_monthly_partition('legal_documents', CURRENT_DATE + INTERVAL '1 month');
```

Troubleshooting común

Síntoma	Diagnóstico	Fix
Queries lentos que antes iban bien	Statistics desactualizadas	`ANALYZE <table>`
Bloat > 50%	Autovacuum no alcanza	VACUUM FULL en ventana, o `pg_repack`
Conexiones saturadas	Pool agotado	Revisar queries lentas; aumentar PgBouncer `default_pool_size`
Writes lentos	Demasiados índices / WAL sync	Revisar necesidad de cada índice; verificar `synchronous_commit`
"Canceling statement due to lock timeout"	DDL en tabla caliente	Aplicar DDL en ventana, usar `CREATE INDEX CONCURRENTLY`